

FreeRTOS基础（八）：FreeRTOS 时间管理

已于 2024-07-19 07:55:31 修改

前面我们用了[FreeRTOS](#)中的延时函数，本篇博客就来探讨FreeRTOS中的延时函数，看看他们是如何发挥作用的。当我们在裸机开发中调用delay_ms()函数时，我们的处理器将不处理任何事，造成处理器资源的浪费。为此，为了提高CPU 的利用率，在FreeRTOS中提供了两个延时API函数。

目录

一、相对延时函数

1.1 函数介绍

1.2 函数实现过程

1.2.1. 任务进入阻塞状态

1.2.2. 计算延迟结束时间

1.2.3. 检查延迟数是否为0

1.2.4. 任务进入延迟列表

1.2.5. 调度任务

1.2.6. 时钟节拍中断处理

1.2.7. 任务唤醒

1.3 总结

二、绝对延时

2.1 函数介绍

2.2 函数实现过程

2.2.1 获取当前时间

2.2.2 计算下一个唤醒时间

2.2.3 检查是否需要延时

2.2.4 更新上一次唤醒时间

2.2.5 返回 pdPASS

2.3 总结

2.4 举例理解

三、相对延时和绝对延时的对比

四、延时函数解析

五、使用延时函数

内容来源：csdn.net

作者昵称：未来可期，静待花开~

原文链接：https://blog.csdn.net/weixin_47040031/article/details/139392173

作者主页：https://blog.csdn.net/weixin_47040031

六、实验验证

6.1 创建任务及实现任务函数

6.2 主函数调用入口函数，操作系统开始进行任务的切换和调度

6.3 实验结果

一、相对延时函数

1.1 函数介绍

FreeRTOS 的相对延时函数 `vTaskDelay` 是一个常用的任务延时函数。**相对延时：指每次延时都是从执行函数 `vTaskDelay()` 开始，直到延时指定的时间结束。**它使当前任务进入阻塞状态一段指定的时间，**以节拍 (ticks) 为单位。**

cpp

AI写代码 | 复制 | 运行

```
1 函数原型: void vTaskDelay( const TickType_t xTicksToDelay );  
2 参数: xTicksToDelay (将调用任务转换到就绪状态前保持在阻塞状态的滴答中断次数)
```

参数

- `xTicksToDelay`: 指定要延迟的时钟节拍数。一个时钟节拍的时间长度取决于系统节拍率 (通常通过 `configTICK_RATE_HZ` 定义)，**我们通常使用的是 1ms 作为一个时钟节拍。**

1.2 函数实现过程

1.2.1. 任务进入阻塞状态

当调用 `vTaskDelay` 时，当前任务会立即进入阻塞状态。该函数首先计算出当前系统的时钟节拍计数 (`xTickCount`) 和需要延迟的节拍数 (`xTicksToDelay`)。

1.2.2. 计算延迟结束时间

内容来源: csdn.net

作者昵称: 未来可期, 静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139392173作者主页: https://blog.csdn.net/weixin_47040031

任务延迟结束的时间点（即节拍计数）由当前节拍计数加上需要延迟的节拍数计算得出：

cppAI写代码 | 复制 运行

```
1 TickType_t xTimeToWake;  
2 xTimeToWake = xTickCount + xTicksToDelay;
```

这里，`xTickCount` 是当前系统的节拍计数，`xTicksToDelay` 是要延迟的节拍数。

1.2.3. 检查延迟数是否为0

如果 `xTicksToDelay` 为 0，函数直接返回，因为不需要延迟。

1.2.4. 任务进入延迟列表

FreeRTOS 使用延迟列表（delay list）来管理处于延迟状态的任务。任务被添加到**延迟列表**中，并记录其唤醒时间（`xTimeToWake`）。

cppAI写代码 | 复制 运行

```
vListInsert( &xDelayedTaskList, &(amp; pxCurrentTCB->xStateListItem ) );
```

1.2.5. 调度任务

任务进入阻塞状态后，调度器会选择下一个任务运行。任务切换是通过 `taskYIELD` 实现的：

cppAI写代码 | 复制 运行

```
taskYIELD();
```

1.2.6. 时钟节拍中断处理

系统时钟节拍中断（tick interrupt）发生时，节拍**计数器** `xTickCount` 递增。中断处理函数 `vTaskIncrementTick` 负责更新系统节拍计数器，并检查是否有任务需要从延迟列表中移到就绪列表。

cppAI写代码 | 复制 运行

```
1 void vTaskIncrementTick( void )  
2 {
```

内容来源: csdn.net
作者昵称: 未来可期，静待花开~
原文链接: https://blog.csdn.net/weixin_47040031/article/details/139892173
作者主页: https://blog.csdn.net/weixin_47040031

```

3   if( uxSchedulerSuspended == ( UBaseType_t ) pdFALSE ) 4 |   {
5       const TickType_t xConstTickCount = xTickCount + ( TickType_t ) 1;
6       xTickCount = xConstTickCount;
7
8       if( xConstTickCount == ( TickType_t ) 0U )
9       {
10          // 检查溢出情况
11      }
12
13      // 检查是否有任务需要唤醒
14      if( xConstTickCount >= xNextTaskUnblockTime )
15      {
16          prvCheckDelayedTasks();
17      }
18  }
19 }

```

1.2.7. 任务唤醒

当当前的节拍计数达到或超过任务的唤醒时间（xTimeToWake）时，延迟任务将从延迟列表中移除，并加入就绪列表。这个过程在 prvCheckDelayedTasks 函数中完成：

```

cpp
1 void prvCheckDelayedTasks( void )
2 {
3     TCB_t *pxTCB;
4
5     while( listLIST_IS_EMPTY( pxDelayedTaskList ) == pdFALSE )
6     {
7         pxTCB = ( TCB_t * ) listGET_OWNER_OF_HEAD_ENTRY( pxDelayedTaskList );
8
9         if( xTickCount < listGET_LIST_ITEM_VALUE( &(amp; pxTCB->xStateListItem) ) )
10        {
11            break;
12        }
13
14        ( void ) uxListRemove( &(amp; pxTCB->xStateListItem) );

```

内容来源: csdn.net

作者昵称: 未来可期, 静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139892173

作者主页: https://blog.csdn.net/weixin_47040031

```
15 | vTaskSwitchContext();16 | }
17 | }
```

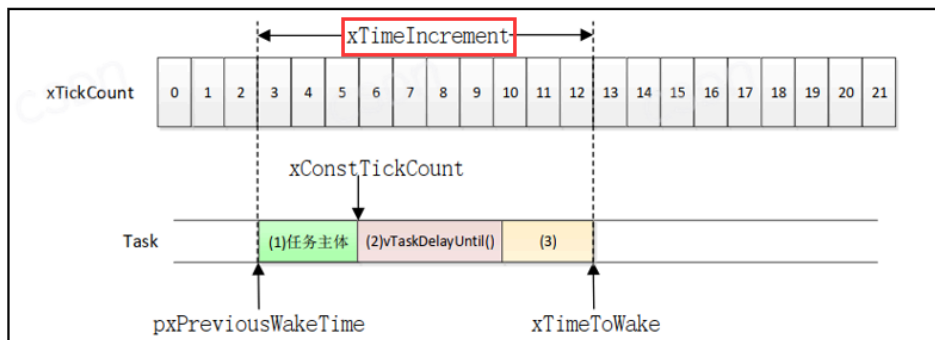
1.3 总结

`vTaskDelay` 函数通过将任务放入延迟列表，计算任务的唤醒时间，并在时钟节拍中断时检查任务是否需要被唤醒，从而实现任务的延时功能。这个过程涉及到任务调度、时钟节拍计数的管理以及列表操作等机制，确保任务能够在指定的时间点后重新进入就绪状态，继续执行。

二、绝对延时

2.1 函数介绍

`xTaskDelayUntil()` 是 FreeRTOS 中用于创建相对精确的周期性任务的函数。它可以确保任务以固定的时间间隔运行，而不受其它任务的影响。与 `vTaskDelay()` 不同，`xTaskDelayUntil()` 通过一个时间基准点来确保周期的准确性。**绝对延时：指将整个任务的运行周期看成一个整体，适用于需要按照一定频率运行的任务。**



(1)为任务主体，也就是任务真正要做的工作

(2)是任务函数中调用`vTaskDelayUntil()`对任务进行延时

(3)为其他任务在运行

CSDN @未来可期，静待花开~

cpp

AI写代码 | 复制 | 运行

```
1 | 函数原型:
2 | BaseType_t xTaskDelayUntil( TickType_t *pxPreviousWakeTime, const TickType_t xTimeIncrement );
```

参数

- `pxPreviousWakeTime`:** 指向一个变量，这个变量保存上一次任务被唤醒的时间。该变量在第一次调用前需要被初始化为当前时间，通常使用 `xTaskGetTickCount()` 获取。

内容来源: csdn.net

作者昵称: 未来可期，静待花开~

作者主页: https://blog.csdn.net/weixin_47040031

- **xTimeIncrement**: 任务之间的时间间隔，即任务周期，单位是 tick 数。

返回值

返回 pdPASS 表示成功。理论上，它总是返回 pdPASS，因此可以忽略返回值。

2.2 函数实现过程

2.2.1 获取当前时间

函数首先获取当前的 tick 计数，即系统运行到目前为止的时间。

cpp

AI写代码 | 复制 | 运行

```
1 TickType_t xTimeNow;  
2 xTimeNow = xTaskGetTickCount();
```

2.2.2 计算下一个唤醒时间

计算任务下一个唤醒时间 xNextWakeTime，这个时间是基于上一次唤醒时间 *pxPreviousWakeTime 加上时间增量 xTimeIncrement。

cpp

AI写代码 | 复制 | 运行

```
1 TickType_t xNextWakeTime;  
2 xNextWakeTime = *pxPreviousWakeTime + xTimeIncrement;
```

2.2.3 检查是否需要延时

比较当前时间 xTimeNow 和下一个唤醒时间 xNextWakeTime，决定是否需要延时。如果当前时间已经超过下一个唤醒时间，说明任务已经错过了预定的周期，此时不会延时；否则，任务需要延时到下一个唤醒时间。

cpp

AI写代码 | 复制 | 运行

```
1 if( xNextWakeTime > xTimeNow ) {  
2     // 延时到下一个唤醒时间  
3     vTaskDelay( xNextWakeTime - xTimeNow );
```

内容来源: csdn.net

作者昵称: 未来可期, 静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139892173

作者主页: https://blog.csdn.net/weixin_47040031

2.2.4 更新上一次唤醒时间

更新 *pxPreviousWakeTime 为 xNextWakeTime, 为下一次调用 xTaskDelayUntil 做准备。

cpp

AI写代码 | 复制 | 运行

```
*pxPreviousWakeTime = xNextWakeTime;
```

2.2.5 返回 pdPASS

函数总是返回 pdPASS 表示成功, 理论上可以忽略返回值。

以下是 xTaskDelayUntil() 的完整实现代码:

cpp

AI写代码 | 复制 | 运行

```
1 BaseType_t xTaskDelayUntil( TickType_t * const pxPreviousWakeTime, const TickType_t xTimeIncrement )
2 {
3     TickType_t xTimeNow;
4     BaseType_t xShouldDelay = pdFALSE;
5
6     configASSERT( pxPreviousWakeTime );
7
8     vTaskSuspendAll();
9     {
10         xTimeNow = xTaskGetTickCount();
11
12         // 下一次唤醒时间
13         const TickType_t xNextWakeTime = *pxPreviousWakeTime + xTimeIncrement;
14
15         // 检查是否需要延时
16         if( xNextWakeTime > xTimeNow ) {
17             xShouldDelay = pdTRUE;
18         }
19     }
20     ( void ) xTaskResumeAll();
21
```

内容来源: csdn.net

作者昵称: 未来可期, 静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139692173作者主页: https://blog.csdn.net/weixin_47040031

```

22 | // 如果需要延时，则调用 vTaskDelay 进行延时
    |
    | 23 | if( xShouldDelay != pdFALSE ) {
24 |     vTaskDelay( xNextWakeTime - xTimeNow );
25 | }
26 |
27 | // 更新上一次唤醒时间
28 | *pxPreviousWakeTime = xNextWakeTime;
29 |
30 | return pdPASS;
31 | }

```

4. 解释

- **vTaskSuspendAll 和 xTaskResumeAll**: 这两个函数用于确保在获取当前时间和计算延时时间的过程中，任务不会被调度器中断。这样可以避免在多任务环境中出现时间不一致的情况。
- **configASSERT**: 这是一个断言，用于检查指针 pxPreviousWakeTime 是否为空，确保函数调用的正确性。
- **时间计算和比较**: 通过 xTimeNow 和 xNextWakeTime 的比较来决定是否需要延时，如果当前时间小于下一个唤醒时间，则需要延时到下一个周期。

2.3 总结

xTaskDelayUntil() 函数通过维护一个上次唤醒时间和一个时间增量，确保任务可以在精确的周期时间内执行。它的实现通过获取当前时间、计算下一个唤醒时间、决定是否需要延时、更新上一次唤醒时间等步骤，确保任务在 FreeRTOS 中能够以固定的时间间隔运行，提高了系统的实时性和任务调度的精确性。

2.4 举例理解

例子 1: 定时闹钟

想象你是一个上班族，每天早上需要在 7:00 起床，你设置了一个闹钟来确保你准时起床。

1. 第一次设定闹钟：你第一次设定闹钟在 7:00。
2. 起床：第二天早上闹钟响了，你在 7:00 起床。
3. 重新设定闹钟：你再次设定闹钟在第二天的 7:00，确保你每天都在同一时间起床。

这里，7:00 就是你的基准时间（`pxPreviousWakeTime`），每天 24 小时的间隔就是时间增量（`xTimeIncrement`）。不管你前一天晚上几点睡觉，闹钟都会确保你第二天早上 7:00 起床。

内容来源：csdn.net

原文链接：https://blog.csdn.net/weixin_47040031/article/details/139392173

作者主页：https://blog.csdn.net/weixin_47040031

例子 2: 公交车的时间表

公交车按照固定的时间表运行, 比如每隔 15 分钟发一班车。

1. 公交车发车: 假设第一班车在早上 8:00 发车。
2. 下一班车时间: 每隔 15 分钟一班, 下一班车在 8:15, 接着是 8:30, 以此类推。

在这个例子中, 早上 8:00 是基准时间 (``pxPreviousWakeTime``), 15 分钟是时间增量 (``xTimeIncrement``)。公交车司机会按照这个固定的时间表发车, 不受前一班车是否晚点的影响。

例子 3: 厨房定时器

你在厨房烘焙蛋糕, 需要每隔 30 分钟检查一次烤箱。

1. 第一次检查: 你在蛋糕入烤箱后的 30 分钟时进行第一次检查。
2. 设定定时器 : 检查完毕后, 你再次设定定时器在接下来的 30 分钟。
3. 后续检查: 每隔 30 分钟你都进行一次检查, 确保蛋糕不会烤焦。

这里, 第一次检查时间就是你的基准时间 (``pxPreviousWakeTime``), 30 分钟是时间增量 (``xTimeIncrement``)。不管你在检查时花了多长时间, 定时器会确保你每隔 30 分钟进行一次检查。

在 FreeRTOS 中的类比

在 FreeRTOS 中, ``xTaskDelayUntil()`` 类似于上述例子中的定时机制, 它保证了任务以固定的时间间隔运行, 而不是简单地延迟一段时间 (这可能会因为任务执行的时间不同而导致整体时间漂移)。

1. 基准时间: 任务上一次唤醒的时间。

2. 时间增量: 任务周期, 即任务之间的固定时间间隔。

每次任务运行完后, 都会计算下次唤醒的时间点 (基准时间 + 时间增量), 然后延时到这个时间点。如果任务执行时间过长导致当前时间已经超过下次唤醒时间, 任务会立即执行而不再延时。

通过 ``xTaskDelayUntil()``, 你可以确保任务以固定的周期运行, 类似于每天固定时间起床、公交车按时间表发车以及厨房定时器检查烤箱。

三、相对延时和绝对延时的对比

内容来源: [csdn.net](https://blog.csdn.net/weixin_47040031)

作者昵称: 未来可期, 静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139392173

作者主页: https://blog.csdn.net/weixin_47040031

对比总结

特性	相对延时 (`vTaskDelay()`)	绝对延时 (`xTaskDelayUntil()`)
延时基准	当前时间	上一次唤醒时间
适用场景	简单的延时任务	需要精确周期执行的任务
实现难度	简单	相对复杂
时间精度	可能不稳定 (受任务执行时间影响)	高 (确保固定时间间隔)
示例应用	一般任务的延时	周期性数据采集、控制循环等
优点	易于使用, 不需要维护额外的时间基准	确保任务以精确的周期运行, 避免时间漂移
缺点	任务执行时间不确定时, 周期可能不稳定	需要额外的代码来维护上一次唤醒时间

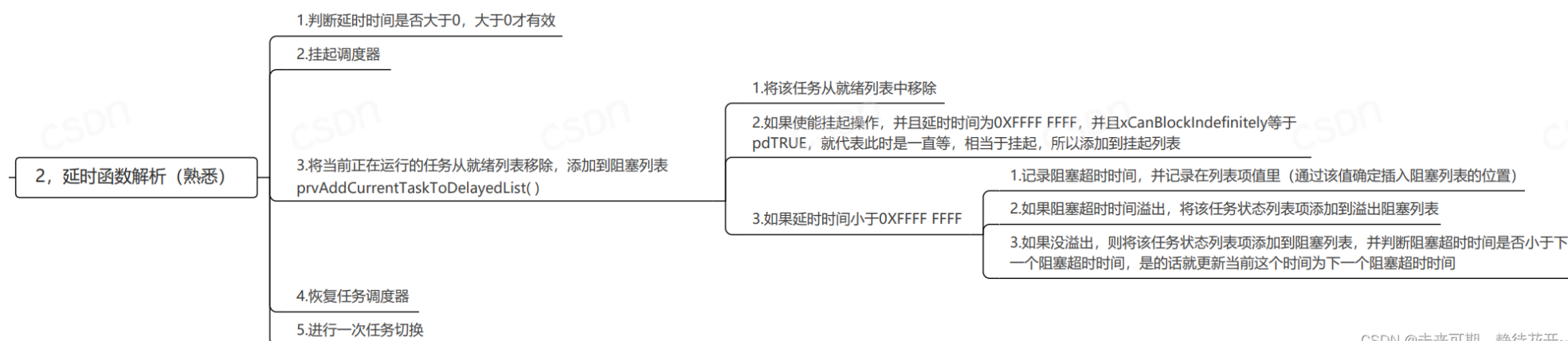
CSDN @未来可期, 静待花开~

适用场景举例

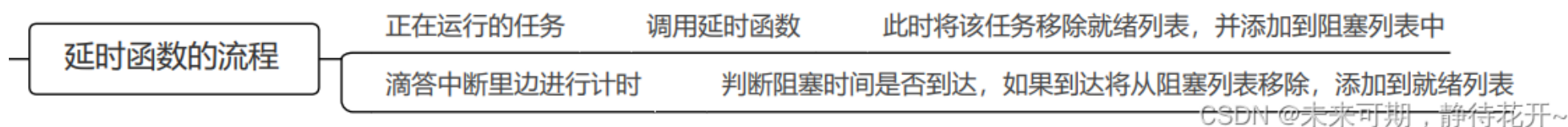
- **相对延时** 适合场景:
 - 简单的 LED 闪烁程序。
 - 需要延时一段时间再执行某些操作, 而不关心时间的绝对精度。
- **绝对延时** 适合场景:
 - 定期采集传感器数据, 每隔固定时间采集一次。
 - 实时控制系统中, 需要以固定时间间隔执行控制算法。

四、延时函数解析

内容来源: csdn.net
作者昵称: 未来可期, 静待花开~
原文链接: https://blog.csdn.net/weixin_47040031/article/details/139392173
作者主页: https://blog.csdn.net/weixin_47040031



CSDN @未来可期, 静待花开~



CSDN @未来可期, 静待花开~

五、使用延时函数

5.1 相对延时函数的使用

使用方式: `vTaskDelay(pdMS_TO_TICKS(100));`或者直接 **`vTaskDelay (毫秒数)`**

`pdMS_TO_TICKS()` 是 FreeRTOS 提供的一个宏, 用于将毫秒数转换为 tick 数。它的定义如下:

```

cpp
AI写代码 | 复制 | 运行

#define pdMS_TO_TICKS( xTimeInMs ) ( ( TickType_t ) ( ( ( TickType_t ) ( xTimeInMs ) * configTICK_RATE_HZ ) / 1000 ) )
  
```

其中 `configTICK_RATE_HZ` 是 FreeRTOS 的 tick 频率配置项, 表示每秒的 tick 数。通过这个宏, 可以方便地将时间转换为 tick 数, 以便使用 `vTaskDelay()` 进行延时。

内容来源: csdn.net
 作者昵称: 未来可期, 静待花开~
 原文链接: https://blog.csdn.net/weixin_47040031/article/details/139392173
 作者主页: https://blog.csdn.net/weixin_47040031

`vTaskDelay()` 的延时精度取决于系统的 tick 频率 (`configTICK_RATE_HZ`)。例如, 如果 tick 频率为 1000 Hz (即 1 ms 一个 tick), 那么延时的精度就是 1 毫秒。如果 tick 频率较低, 延时的精度也会降低。

5.2 绝对延时函数的使用

1、初始化基准时间

在任务函数中, 使用 `xTaskGetTickCount()` 获取当前的 tick 计数, 并将其赋值给 `xLastWakeTime`, 作为初始的基准时间。

cpp

AI写代码 | 复制 | 运行

```
1 | TickType_t xLastWakeTime = xTaskGetTickCount(); // 获取当前的tick数
2 | TickType_t xFrequency = pdMS_TO_TICKS(1000); // 延时时间为1秒周期
3 |
```

2、进行绝对延时

cpp

AI写代码 | 复制 | 运行

```
vTaskDelayUntil(&xLastWakeTime, xFrequency);
```

六、实验验证

定义三个任务, 任务1和任务2使用`vTaskDelay()`函数。任务3使用`vTaskDelayUntil()`函数。任务3将以固定的频率进行执行。

6.1 创建任务及实现任务函数

cpp

AI写代码 | 复制 | 运行

```
1 | #include "stm32f4xx.h" // Device header
2 | #include "stdio.h"
3 | #include "FreeRTOS.h"
4 | #include "task.h"
5 | #include "dynamic.h"
6 |
```

内容来源: csdn.net

作者昵称: 未来可期, 静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139892173

作者主页: https://blog.csdn.net/weixin_47040031

```

7
8 |
9 /*****START_TASK任务配置*****/
10 /*****包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
11
12 #define START_TASK_STACK_SIZE 128 //定义堆栈大小为128字（1字等于4字节）
13 #define START_TASK_PRIO 1 //定义任务优先级，0-31根据任务需求
14 TaskHandle_t start_task_handler; //定义任务句柄（结构体指针）
15 void start_task(void* args);
16
17
18
19
20 /*****TASK1任务配置*****/
21 /*****包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
22 #define TASK1_STACK_SIZE 128 //定义堆栈大小为128字（1字等于4字节）
23 #define TASK1_PRIO 1 //定义任务优先级，0-31根据任务需求
24 TaskHandle_t task1_handler; //定义任务句柄（结构体指针）
25 void task1(void* args);
26
27
28
29
30 /*****TASK2任务配置*****/
31 /*****包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
32 #define TASK2_STACK_SIZE 128 //定义堆栈大小为128字（1字等于4字节）
33 #define TASK2_PRIO 1 //定义任务优先级，0-31根据任务需求
34 TaskHandle_t task2_handler; //定义任务句柄（结构体指针）
35 void task2(void* args);
36
37
38
39 /*****TASK3任务配置*****/
40 /*****包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
41 #define TASK3_STACK_SIZE 128 //定义堆栈大小为128字（1字等于4字节）
42 #define TASK3_PRIO 2 //定义任务优先级，0-31根据任务需求
43 TaskHandle_t task3_handler; //定义任务句柄（结构体指针）
44 void task3(void* args);
45

```

内容来源: [csdn.net](https://blog.csdn.net/weixin_47040031)

作者昵称: 未来可期, 静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139692173

作者主页: https://blog.csdn.net/weixin_47040031

```

46
47 |
48
49
50
51 /***** 开始任务用来创建其他三个任务，只创建一次，不能是死循环，同时创建完3个任务后删除任务1本身 *****/
52 void start_task(void* args)
53 {
54     taskENTER_CRITICAL(); /*进入临界区*/
55     BaseType_t xReturn;
56
57     xTaskCreate( (TaskFunction_t)      task1,
58                 (char *)              "task1",
59                 ( configSTACK_DEPTH_TYPE) TASK1_STACK_SIZE,
60                 (void *)              NULL,
61                 (UBaseType_t)          TASK1_PRI0 ,
62                 (TaskHandle_t *)       &task1_handler );
63
64     /*对任务1创建是否成功进行判断*/
65     if(xReturn == pdPASS)
66     {
67         printf("create task1 Success\n");
68     }
69     else
70     {
71         printf("create task1 error\n");
72     }
73
74
75     xTaskCreate( (TaskFunction_t)      task2,
76                 (char *)              "task2",
77                 ( configSTACK_DEPTH_TYPE) TASK2_STACK_SIZE,
78                 (void *)              NULL,
79                 (UBaseType_t)          TASK2_PRI0 ,
80                 (TaskHandle_t *)       &task2_handler );
81
82     /*对任务2创建是否成功进行判断*/
83     if(xReturn == pdPASS)
84     {

```

内容来源: csdn.net

作者昵称: 未来可期，静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139692173

作者主页: https://blog.csdn.net/weixin_47040031

```

85     printf("create task2 Success\n"); 86 }
86
87 else
88 {
89     printf("create task2 error\n");
90 }
91
92
93     xTaskCreate( (TaskFunction_t)      task3,
94                 (char *)              "task3",
95                 ( configSTACK_DEPTH_TYPE) TASK3_STACK_SIZE,
96                 (void *)              NULL,
97                 (UBaseType_t) TASK3_PRIO ,
98                 (TaskHandle_t *)      &task3_handler );
99
100     /*对任务3创建是否成功进行判断*/
101     if(xReturn == pdPASS)
102     {
103         printf("create task3 Success\n");
104     }
105     else
106     {
107         printf("create task3 error\n");
108     }
109
110
111     vTaskDelete(NULL);    //删除开始任务自身，传参NULL
112
113     taskEXIT_CRITICAL();  /*退出临界区*/
114
115
116     //临界区内不会进行任务的调度切换，出了临界区才会进行任务调度，抢占式
117 }
118
119
120
121
122     /******* 其余三个任务的任务函数，无返回值且是死循环***** */
123
124     /*** 任务1：打印字符串验证***/

```

内容来源: csdn.net

作者昵称: 未来可期，静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139692173

作者主页: https://blog.csdn.net/weixin_47040031

```

125 void task1(void* args)
126 {
127     while(1)
128     {
129         printf("任务1运行! \n");
130         vTaskDelay(300); //FreeRTOS自带的延时函数
131     }
132 }
133
134
135 }
136
137 /**任务2：打印字符串验证***/
138 void task2(void* args)
139 {
140     while(1)
141     {
142         printf("任务2运行! \n");
143         vTaskDelay(300); //FreeRTOS自带的延时函数
144     }
145 }
146 }
147
148
149 /**任务3：实现绝对延时***/
150 void task3(void* args)
151 {
152     TickType_t xLastWakeTime = xTaskGetTickCount(); //获取当前的tick数
153     TickType_t xFrequency = pdMS_TO_TICKS(2000); //延时时间为2秒周期
154     while(1)
155     {
156         printf("任务3运行! \n");
157         vTaskDelayUntil(&xLastWakeTime, xFrequency);
158     }
159 }
160 }
161
162
163

```

内容来源: [csdn.net](https://blog.csdn.net/weixin_47040031)

作者昵称: 未来可期, 静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139692173

作者主页: https://blog.csdn.net/weixin_47040031

```

164 | 165 |
166 | //FreeRTOS入口例程函数，无参数，无返回值
167 | void freertos_demo(void)
168 | {
169 |     xTaskCreate( (TaskFunction_t) start_task,
170 |                 (char *) "start_task",
171 |                 ( configSTACK_DEPTH_TYPE) START_TASK_STACK_SIZE,
172 |                 (void *) NULL,
173 |                 (UBaseType_t) START_TASK_PRIO ,
174 |                 (TaskHandle_t *) &start_task_handler );
175 |
176 |
177 |     vTaskStartScheduler(); //开启任务调度器
178 |
179 | }
180 |
181 |
182 |
183 |

```

6.2 主函数调用入口函数，操作系统开始进行任务的切换和调度

cpp

AI写代码 | 复制 | 运行

```

1 | #include "stm32f4xx.h" // Device header
2 | #include "stdio.h"
3 | #include "myusart.h"
4 |
5 | #include "FreeRTOS.h"
6 | #include "task.h"
7 | #include "dynamic.h"
8 |
9 |
10 |
11 | int main(void)
12 | {
13 |     //1、硬件初始化
14 |     My_UartInit();

```

内容来源: csdn.net

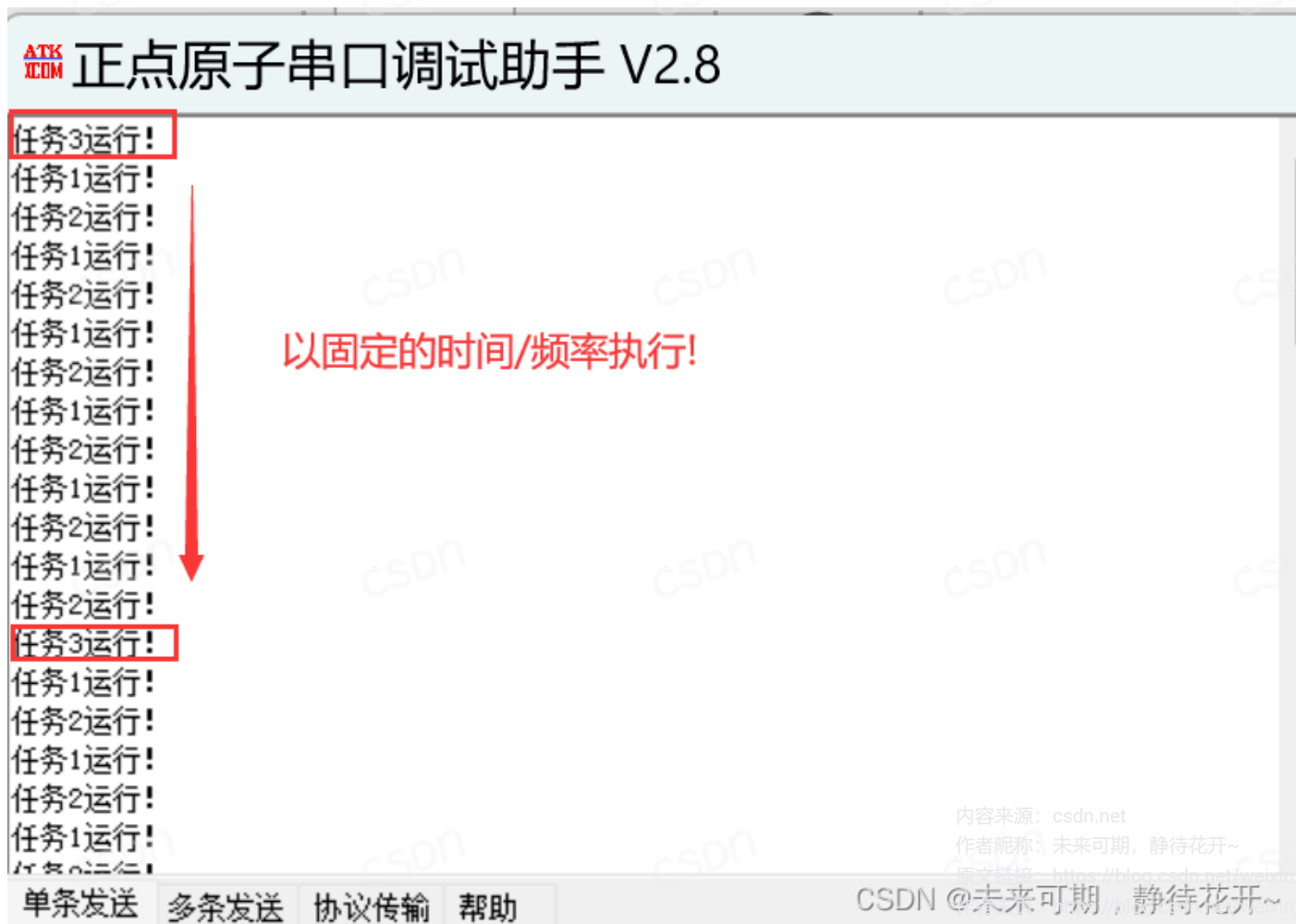
作者昵称: 未来可期, 静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139692173

作者主页: https://blog.csdn.net/weixin_47040031

```
15 |  
16 | //2、进入入口函数  
17 | freertos_demo();  
18 |  
19 |  
20 |  
21 | }  
22 |  
23 |  
24 |
```

6.3 实验结果



至此，已经讲解完毕！初次学习，循序渐进，一步步掌握即可！以上就是全部内容！请务必掌握，创作不易，欢迎大家点赞加关注评论，您的支持是我前进最大的动力！下期再见！

标签

#单片机

#FreeRTOS

内容来源：[csdn.net](https://blog.csdn.net/weixin_47040031)

作者昵称：未来可期，静待花开~

原文链接：https://blog.csdn.net/weixin_47040031/article/details/139392173

作者主页：https://blog.csdn.net/weixin_47040031